

Tenbin Security Review — Consolidated Findings and Remediation Brief

Pre-remediation report

Client	Tenbin Labs
Date	2026-08-15
Solidity	0.8.24 / 0.8.27 / 0.8.30
Audit posture	Report-only. No repository code was modified. No chain state was read.
Status	Pre-remediation. Two severities pending client determination.

 High	 Medium	 Low	 Info	Total
2 ‡	5	7	5	19

‡ **H-02** is rated High or Low pending the determination in §3. If Low, the distribution is High 1 / Medium 5 / Low 8 / Info 5.

Six further observations are recorded in §7 as **N-2** through **N-8**; they are not counted as findings.

1. Purpose and status

This report presents all findings from the security review, with remediation guidance and stated acceptance criteria for each. It is issued for remediation purposes and is **not the final public report**, which will be produced after fixes are applied and will reflect the remediated code.

It consolidates two prior working documents — the findings consolidation (`tenbin-security-audit-consolidated.md`) and the remediation brief (`TENBIN_REMEDIATION_BRIEF_2026-08-15.md`) — into a single delivery. Both documents used independent identifier schemes; every finding below carries its prior identifiers, and a full crosswalk is provided in §4 and Appendix A. Severities marked as conditional depend on

client determinations recorded in §10.

Summary of exposure. No critical findings were identified. One High finding (**H-01**) is unconditional and is not patchable after deploy. A second (**H-02**) is conditional on a scope determination. Risk is otherwise concentrated in three areas:

- **CrossChainERC4626Adapter** , whose forked two-message cooldown flow introduces the first class of user value the contract custodies across transactions, gates release of that value on controls designed for new business, and — where a delivery fails with tokens attached and no local refund address — retains no on-chain exit for those tokens at all.
- **GenericMorphoAdapter** , whose supported target set was widened from MetaMorpho to “any standard ERC4626” while three upstream protections were removed. Six findings originate here, including two of the five Mediums.
- **Payload and encoding drift from the Legacy fork**, where an unchanged `typeAndVersion` string and a reinterpreted first payload word leave Tenbin and Legacy adapters indistinguishable to integrators.

Where the two source documents assigned different severities to the same finding, the divergence is stated explicitly in the finding body under **Severity reconciliation** rather than silently resolved.

2. Scope

Repo	Commit
<code>cross-chain-erc4626-vault</code>	<code>cde2faffec4470c9a3f33103efd6a52d027cd696</code>
<code>tenbin-contracts</code>	<code>633540ee51d7a069f4a5a53feccd8e3498d1ff14</code>

Components reviewed

Component	Path
Cross-chain adapter	<code>cross-chain-erc4626-vault/src/CrossChainERC4626Adapter.sol</code>
Staked asset vault	<code>tenbin-contracts/src/StakedAsset.sol</code>
Bebop hook	<code>tenbin-contracts/src/.../BebopHook.sol</code> , <code>IBebopHook.sol</code>

Morpho generic adapter	tenbin-contracts/src/external/morpho/GenericMorphoAdapter.sol
Morpho adapter factory	tenbin-contracts/src/external/morpho/GenericMorphoAdapterFactory.sol
Morpho gate	tenbin-contracts/src/external/morpho/Gate.sol
Frontend — send flow	cross-chain-erc4626-vault/frontend/client/src/components/vault/SendDialog.tsx
Frontend — message tracking	cross-chain-erc4626-vault/frontend/client/src/components/vault/TrackedMessages.tsx

Referenced as dependencies but not themselves in scope: `VaultV2.sol` , `CollateralManager.sol` , `Deploy.s.sol` , and the vault configuration files under `config/` .

Every statement regarding what is deployed, configured, or current is a claim about the repository snapshot at the commits above.

3. Unresolved scope determination

The scope document describes the objective as supporting “a standard, **non-synchronous ERC4626**”, and uses that phrase twice. The severity of **H-02** depends on whether this is literal:

Reading	H-02 severity	Basis
Literal — asynchronous and cooldown-gated children are the intended target class	High	The adapter cannot perform its stated function, and the failure mode is a successful deposit followed by permanent inability to withdraw
Typographical — intended as <i>synchronous</i> ; the adapter targets only same-transaction ERC-4626 vaults	Low	Documentation and target-admission gap

All code facts underlying the finding are identical under both readings and are not in

question. A determination is required from Tenbin; this review will not substitute one. The determination also governs the scope of remediation item 1 in §5.

Note that the earlier findings consolidation rated this condition **Informational** (as I-05), on the basis that the failed call is clean and balances are unchanged. That assessment did not account for the irreversibility of the funded position — see H-02, *Severity reconciliation*.

4. Index

ID	Severity	Title	Prior IDs
H-01	 High	Failed CCIP messages can permanently lock delivered ERC-20s	H-01
H-02	 High or Low ‡	Synchronous-only deallocation locks capital in a cooldown or asynchronous child	GMA-4 , I-05
M-01	 Medium	Inbound payload first word is <code>actionAndId</code> ; Legacy encoding is misread	M-01
M-02	 Medium	Zero beneficiary passes validation and can lose bridged funds	M-02
M-03	 Medium	Committed cooldown principal is gated on entry-side switches, with no escape hatch	CCA-1 , I-01 , I-06
M-04	 Medium	Fee-exclusive marking permits externalization of the child's exit fee	GMA-2a
M-05	 Medium	Donated shares count at zero allocation and can never be permissionlessly removed	GMA-2b
L-01	 Low	Failed <code>UNSTAKE</code> / <code>CANCEL_COOLDOWN</code> messages stay permanently unresolved	L-01 , N1
L-02	 Low	Solana path always assumes 9 decimals (USDC is 6)	L-02
L-03	 Low	A single static <code>liquidityData</code> blob cannot serve both allocation directions	GMA-1 , M-03
L-		Permissionless dust returns can exhaust the shared	

04	 Low	native CCIP reserve	CCA-3
L-05	 Low	Inflation-resistance requirement removed while the target set was widened	GMA-3
L-06	 Low	Staked-asset redeem passes validation but can never execute	BH-1
L-07	 Low	Caller-chosen <code>owner</code> defeats the write-once registry guarantee	GMAF-1 , I-07
I-01	 Info	Gate manager can be <code>address(0)</code> , which locks the vault	I-02
I-02	 Info	Unused fee roles and outdated docs left after the Tenbin fork	I-03 , N5
I-03	 Info	UI shows Success when the vault action actually failed	I-04
I-04	 Info	Fail-open cooldown authorization bypasses the file's own conversion rail	CCA-2
I-05	 Info	Each order signer must separately call <code>setRecipientStatus</code>	CTRL-1

Crosswalk from prior documents

Both source documents are superseded by this one. Neither prior identifier scheme survives intact; use the table below when reconciling against earlier correspondence.

Findings consolidation	Remediation brief	This report
H-01	—	H-01
I-05	GMA-4	H-02 (escalated)
M-01	—	M-01
M-02	—	M-02
I-01 + I-06	CCA-1	M-03 (escalated, merged)
—	GMA-2a	M-04

—	GMA-2b	M-05
L-01	N1	L-01 (<i>merged</i>)
L-02	—	L-02
M-03	GMA-1	L-03 (<i>de-escalated</i>)
—	CCA-3	L-04
—	GMA-3	L-05
—	BH-1	L-06
I-07	GMAF-1	L-07 (<i>escalated</i>)
I-02	—	I-01
I-03	N5	I-02 (<i>merged</i>)
I-04	—	I-03
—	CCA-2	I-04
—	CTRL-1	I-05
—	N2 – N4 , N6 – N8	§7 observations

Finding relationships

Several findings interlock; fixing one changes the practical severity of another.

- **H-01 ← M-01, M-03.** H-01 is the loss primitive (stranded ERC-20s). M-01 and M-03 are two of the realistic ways to *reach* it — a Legacy-encoded send that reverts on `InvalidAction` , and a token-carrying `DEPOSIT` arriving while `depositsEnabled` is false. Fixing H-01's recovery path defuses both.
- **H-01 ↔ I-03.** A user whose tokens are permanently stranded sees a green success indicator, delaying the support escalation that might otherwise trigger a manual refund attempt while the return lane is still healthy.
- **L-01 ↔ M-03.** The proof of concept recorded under M-03 also demonstrates L-01's mechanic: the failed tokenless `UNSTAKE` becomes `BASIC` and can never reach `RESOLVED` . They remain separate findings because the fixes differ — L-01 is bookkeeping hygiene, M-03 is access-control design.

- **M-03 ↔ L-04.** A cooldown initiated while the native reserve is healthy commits the system to a later tokenless exit message. Where the reserve is exhausted by that point, the exit cannot be paid — the same trapped-principal condition, reached economically rather than through operator action.
- **M-04 ↔ M-05.** Both are `realAssets()` valuation defects and are resolved by a single edit. That edit has a naive form which creates a fund lock; see M-05 before implementing either.
- **L-03 ↔ L-05.** The unbounded automatic-allocation path in L-03 is the exposure surface that makes L-05's sandwich reachable at all; the explicit allocator path is protected by `minShares`.
- **L-03 ↔ H-02.** Both are `GenericMorphoAdapter` integration constraints that surface only against particular parent/child vault pairings.
- **M-01 ↔ I-02.** Both are fork residue that misleads integrators — an unchanged `typeAndVersion` string, and dead fee roles that survive the removal of the fee mechanism.

5. Remediation priority

#	Action	Findings closed	Effort
1	Add a scoped recovery path for token-carrying <code>BASIC</code> messages with a zero <code>localRefundAddress</code>, and require a non-zero address at the encoder	H-01	Medium
2	Resolve the scope determination in §3	H-02	—
3	Split <code>_position()</code> / guarded <code>realAssets()</code> in <code>GenericMorphoAdapter</code>	M-04, M-05	Small
4	Decouple cooldown settlement from entry-side switches; add a beneficiary-pinned escape hatch	M-03	Medium
5	Move <code>actionAndId</code> into the first payload field on every sender; change <code>typeAndVersion</code>	M-01	Small
6	Reject a zero beneficiary on both chain families, including SVM <code>tokenReceiver</code>	M-02	Trivial

7	Restore target-admission requirements; add the <code>shares == 0 guard</code>	L-05, H-02	Small
8	Remove the <code>owner</code> parameter from the factory	L-07	Small
9	Make <code>data</code> direction-agnostic, or prohibit the adapter as <code>liquidityAdapter</code>	L-03	Small
10	Reject the staked-asset redeem shape at validation	L-06	Small
11	Require prepaid outbound quotes; reject dust returns	L-04	Medium
12	Read real SPL token decimals in the Solana send path	L-02	Small
13	Add a resolve path for tokenless <code>BASIC</code> records	L-01	Small
14	Normalize the cooldown sender; add the <code>bytes32(0) guard</code>	I-04	Trivial
15	Guard <code>setManager</code> against zero; read adapter status before showing UI success	I-01, I-03	Trivial
16	Correct the <code>max_rate</code> comment in all seven configs; add <code>setRecipientStatus</code> to the runbook; remove dead fee roles	N-8, I-05, I-02	Trivial

Item 1 is deploy-blocking. H-01 cannot be patched after deploy — `ROUTER`, `VAULT`, and `ASSET` are immutable and the adapter is not upgradeable. Every other item on this list remains addressable, in some form, post-deploy.

Item 3 is the highest-leverage code change: a single edit resolves both Medium findings in the adapter. The naive form of that edit introduces a fund lock; see M-05 before implementation.

6. Findings

High

[H-01](#)

H-01 | High

Failed CCIP messages can permanently lock delivered ERC-20s

Field	Value
Severity	High
Source	cross-chain-erc4626-vault/src/CrossChainERC4626Adapter.sol
ID	H-01
Prior IDs	H-01 (findings consolidation)
Commit	cde2faffec4470c9a3f33103efd6a52d027cd696
Status	Pending

■ Description

The adapter is not upgradeable. `ROUTER`, `VAULT`, and `ASSET` are immutable. When the CCIP router has already credited tokens and `processMessage` then reverts, `ccipReceive` catches the error, marks the message `BASIC`, and returns success to the router — so CCIP will not redeliver.

The only exits are `refundFailedMessage` (bridge back to the original sender) and `recoverFailedMessageLocally` (only if the payload stored a non-zero `localRefundAddress`). There is no in-place retry and no admin ERC-20 recovery; `recoverNative` moves ETH only.

If `localRefundAddress` is zero — because the payload field was left blank, an invalid address was coerced to zero, or the payload length $\neq 128$ so the field is never unpacked — and the return lane cannot send, those tokens stay in the adapter forever.

Token-carrying failures with no local refund address have no on-chain ERC-20 exit when the return bridge also fails. **This cannot be patched after deploy.**

Example. A user bridges 100 USDC while `depositsEnabled` is false. Tokens arrive; processing reverts; the message is marked `BASIC`. `localRefundAddress` was left as `address(0)`. If the source-chain CCIP lane cannot refund, the 100 USDC has no recovery path.

The enumeration of movement sites recorded under M-03 corroborates this independently: the file contains four ERC-20 movement sites, of which only `:555` (inbound delivery) and `:766` (failed-message local recovery) move value out. The fork deleted `withdrawFee`, the legacy contract's only other operator-directed transfer.

■ Impact

User principal can be locked in the adapter with no admin or user function able to move it. Any later adapter deploy cannot reach these balances.

■ Code

CrossChainERC4626Adapter.sol :

```
function recoverNative(address recipient) external onlyRole(DEFAULT_ADMIN_ROLE) {
    // ETH only – cannot move failed-message ERC-20s
    recipient.call{value: address(this).balance}("");
}

function recoverFailedMessageLocally(bytes32 messageId) external nonReentrant {
    address recipient = record.localRefundAddress;
    if (recipient == address(0)) revert NoLocalRefundAddress(messageId);
    if (msg.sender != recipient) revert UnauthorizedLocalRefund(msg.sender, recip
    if (!_anyPositiveDestAmount(record.destTokenAmounts)) revert NoRefundableToke
    // ...
}

function _storeFailedMessageRecord(bytes32 messageId, Client.Any2EVMMessage calld
    record.destTokenAmounts = message.destTokenAmounts;
    if (message.data.length == CCIP_MESSAGE_PAYLOAD_LENGTH) {
        (, address localRefundAddress) = _unpackDeliveryAndRefund(payload.deliver
        record.localRefundAddress = localRefundAddress; // may be 0
    }
    // length != 128 → localRefundAddress stays address(0)
}
```

■ Recommendation

Before deploy, add a scoped recovery path for `BASIC` messages whose

`localRefundAddress` is zero — for example an admin escrow that can receive only that message's `destTokenAmounts`. **Do not** add a blanket `rescueERC20`. Additionally, require a non-zero `localRefundAddress` on every token-carrying send at the encoder.

After deploy, this is not fixable on the live adapter.

Suggested fix — CrossChainERC4626Adapter.sol :

```
address public defaultLocalRefund; // set at deploy; never zero

function setDefaultLocalRefund(address account) external onlyRole(DEFAULT_ADMIN_R
    if (account == address(0)) revert InvalidRecipient();
```

```

        defaultLocalRefund = account;
    }

    function recoverFailedMessageLocally(bytes32 messageId) external nonReentrant {
        if (messageErrorCode[messageId] != ErrorCode.BASIC) revert MessageNotFailed(m
        FailedMessageRecord memory record = failedMessageRecords[messageId];
        if (!_anyPositiveDestAmount(record.destTokenAmounts)) revert NoRefundableToke

        address recipient = record.localRefundAddress;
        if (recipient == address(0)) {
            recipient = defaultLocalRefund;
            if (recipient == address(0)) revert NoLocalRefundAddress(messageId);
            if (!hasRole(DEFAULT_ADMIN_ROLE, msg.sender))
                revert UnauthorizedLocalRefund(msg.sender, recipient);
        } else if (msg.sender != recipient) {
            revert UnauthorizedLocalRefund(msg.sender, recipient);
        }

        messageErrorCode[messageId] = ErrorCode.RESOLVED;
        delete failedMessageRecords[messageId];
        delete refundChainFamilySnapshot[messageId];
        for (uint256 i = 0; i < record.destTokenAmounts.length; ++i) {
            uint256 amt = record.destTokenAmounts[i].amount;
            if (amt == 0) continue;
            IERC20(record.destTokenAmounts[i].token).safeTransfer(recipient, amt);
        }
        emit MessageRecoveredLocally(messageId, recipient);
    }
}

```

■ Acceptance criteria

- A test that delivers a token-carrying message which fails processing with `localRefundAddress == address(0)`, and demonstrates the tokens are recoverable through the scoped path.
- A test that the scoped path can move only that message's `destTokenAmounts`, and cannot reach unrelated adapter balances.
- An encoder-side test that a token-carrying send with a zero `localRefundAddress` is rejected before dispatch.

■ Status

Pending

[↑ Back to index](#)

H-02 | ■ High or ■ Low [#]

Synchronous-only deallocation locks capital in a cooldown or asynchronous child

Field	Value
Severity	High or Low — pending the determination in §3
Source	tenbin-contracts/src/external/morpho/GenericMorphoAdapter.sol , tenbin-contracts/src/StakedAsset.sol
ID	H-02
Prior IDs	GMA-4 (remediation brief), I-05 (findings consolidation)
Commit	633540ee51d7a069f4a5a53feccd8e3498d1ff14
Status	Pending

■ Description

`deallocate()` exits exclusively through the child's synchronous `withdraw()` , and VaultV2 requires the assets to arrive in the same transaction. A child implementing the ERC-4626 interface but gating exit behind a cooldown or a request/claim cycle — the ERC-7540 shape — accepts allocations and refuses every exit.

On `StakedAsset` , if `cooldownPeriod > 0` , `withdraw()` reverts `RequiresCooldown` . The call fails cleanly — shares and assets are neither burned nor moved — but the adapter has no cooldown or unstake path, so the pull-back stays blocked until `cooldownPeriod` is set to 0 or the position is exited by some other route.

The condition is not detectable at configuration time. It manifests only after the position is funded, at which point:

- every allocator `deallocate()` reverts;
- every permissionless `forceDeallocate()` reverts;
- the child shares are explicitly non-rescuable (:126), and `vault` is immutable at :40 .

No configuration change, timelocked repair, or administrative action recovers the position.

The misconfiguration is irreversible.

The affected target class is common — a cooldown-gated vault exposing the ERC-4626 interface and reverting on synchronous exit — and `StakedAsset`, in this repository, is an instance of it.

■ Severity reconciliation

The findings consolidation rated this **Informational**, reasoning that “balances are unchanged on the failed call, so nothing is lost or corrupted — this is a pairing constraint, not a bug in either contract individually.”

That reasoning is correct about the *call* and incorrect about the *position*. The revert is clean, but the funded state that precedes it is not recoverable: `vault` is immutable, child shares are non-rescuable, and both the allocator and permissionless exit valves revert. An Informational rating is appropriate only if the pairing is out of scope by construction — which is precisely the determination sought in §3.

■ Impact

Under the literal reading of the scope wording, the adapter cannot perform its stated function, and the failure mode is a successful deposit followed by permanent inability to withdraw. Under the typographical reading, the exposure is a documentation and target-admission gap: the adapter works correctly against synchronous children, and the hazard is that nothing prevents an operator from pointing it at an asynchronous one.

In either case the parent vault cannot pull those assets back through this adapter while cooldown is required, and the inflated or stranded position propagates into Tenbin’s collateral accounting.

■ Code

`GenericMorphoAdapter.sol` :

```
function deallocate(...) external returns (bytes32[] memory, int256) {
    if (assets > 0) {
        shares = vault.withdraw(assets, address(this), address(this));
        if (shares > maxShares) revert ExcessiveShares();
    }
}
```

`StakedAsset.sol` :

```
function withdraw(uint256 assets, address receiver, address owner) public override
    if (cooldownPeriod > 0) revert RequiresCooldown();
    return super.withdraw(assets, receiver, owner);
}
```

■ Recommendation

1. Prohibit allocation to any cooldown or asynchronous child under this implementation, stated in the NatSpec at :12-16 alongside the seeding requirement L-05 requires restored.
2. Add a deployment-time compatibility check where feasible — a `vault.maxWithdraw(address(this))` probe for a nonzero amount, or an explicit allowlist of vetted target shapes in the factory. Optionally, have the factory warn on or reject the `StakedAsset` pairing at deployment time.
3. If asynchronous targets are in scope, a purpose-built adapter is required: it must initiate exit requests, track request identifiers and ownership, and claim matured assets into an adapter-side liquid balance. `realAssets()` must report the fee-aware NAV of all economically owned pending and claimed positions, and immediately transferable liquidity must be maintained separately for `deallocate()`. A genuinely asynchronous child cannot serve as a just-in-time liquidity adapter without pre-requested exits or a liquid reserve; this constraint shapes the design and should be settled before implementation begins.

■ Acceptance criteria

- NatSpec states the restriction explicitly, or a compatibility check rejects asynchronous targets at deployment.
- Where an asynchronous adapter is built: an integration test demonstrating a complete `allocate → request → claim → deallocate` lifecycle, and a test that `deallocate()` succeeds against a child whose exit is queued.

■ Status

Pending — severity conditional on §3

[↑ Back to index](#)

M-01 | Medium

Inbound payload first word is `actionAndId` ; Legacy encoding is misread

Field	Value
Severity	Medium
Source	<code>cross-chain-erc4626-vault/src/CrossChainERC4626Adapter.sol</code>
ID	M-01
Prior IDs	M-01 (findings consolidation)
Commit	<code>cde2faffec4470c9a3f33103efd6a52d027cd696</code>
Status	Pending

■ Description

The Tenbin adapter binds a single immutable `VAULT`. The first payload word is `actionAndId` (action in bits 128–135, cooldown id in the low 128 bits), **not** a vault address. Legacy clients still encode `address target` in that slot.

Both layouts are 128 bytes, so the adapter accepts the message and decodes the address as an action. `uint8(address >> 128)` is one byte of the vault address:

- Value `0-3` → runs the **wrong action** against `VAULT`.
- Value `>3` → reverts `InvalidAction` and the message becomes `BASIC` — which is H-01 if tokens were delivered.

`typeAndVersion` is still `"CrossChainERC4626Adapter 1.0.0"`, identical to Legacy, so clients cannot distinguish the two. This ABI cannot be extended on the live contract.

A Legacy-encoded send to a Tenbin adapter does not target a chosen vault. It either runs a garbage action or fails and can strand tokens.

Example. The UI encodes `[vault address][beneficiary][minimumOut][deliveryAndRefund]` and sends shares for a redeem. Tenbin reads the action from a byte of that address. Most addresses yield `InvalidAction` → `BASIC`, with shares sitting on the adapter.

Related: bits 136–255 of `actionAndId` are neither read nor validated (:516–518), recorded separately as observation **N-4**.

■ Impact

Wrong vault action, or a failed delivery that can strand tokens via H-01. This occurs whenever any Legacy encoder talks to this adapter — including the `payload.ts` and deploy scripts still present in this repo.

■ Code

`CrossChainERC4626Adapter.sol` :

```
struct Payload {
    uint256 actionAndId; // bits 128+ = action 0..3; low 128 = cooldown id
    bytes32 beneficiary;
    uint256 minimumOut;
    uint256 deliveryAndRefund;
}

uint8 vaultAction = uint8(payload.actionAndId >> 128);
if (vaultAction > 3) revert InvalidAction();
// Legacy first word was an address → this byte is not an action
```

■ Recommendation

Update every sender (UI, tests, partner integrations) to place `actionAndId` in the first payload field rather than a vault address. Change `typeAndVersion` so Tenbin and Legacy adapters are not labeled identically — this is the only durable signal available to a third-party integrator, and it costs nothing.

Validate or explicitly reserve bits 136–255 rather than ignoring them.

■ Acceptance criteria

- A test that a Legacy-shaped payload is rejected with a typed error rather than silently decoded as an action.
- `typeAndVersion` differs from the Legacy adapter's string, asserted in a test.
- Every encoder in the repository (`payload.ts` , deploy scripts, tests) produces the Tenbin layout, verified by a shared fixture.

■ Status

Pending

[↑ Back to index](#)

M-02 | Medium

Zero beneficiary passes validation and can lose bridged funds

Field	Value
Severity	Medium
Source	<code>cross-chain-erc4626-vault/src/CrossChainERC4626Adapter.sol</code>
ID	M-02
Prior IDs	M-02 (findings consolidation)
Commit	<code>cde2faffec4470c9a3f33103efd6a52d027cd696</code>
Status	Pending — see determination 5 in §10

■ Description

Listed in case a zero beneficiary is not intended.

`_requireValidEvmBeneficiary` only checks that the high 96 bits of the `bytes32` are zero — that is, correct packing. It still accepts `bytes32(0) / address(0)` .

- **Local delivery:** many ERC-20s revert on transfer to zero, so the message fails → `BASIC` .
- **EVM return-to-source:** the same packing check runs before encoding the receiver.
- **SVM branch:** the check is skipped entirely and `beneficiary` is passed straight into `SVMExtraArgsV1.tokenReceiver` — so a zero Solana receiver is accepted with no validation at all.

The on-chain check validates packing, not non-zeroness. If a zero beneficiary is not intended, the SVM return path is the one that can burn funds.

This is the same class of defect as I-04 — a `bytes32` whose zero value is not rejected where it should be — and both should be fixed together.

■ Impact

If zero is allowed by design, no change is needed. If it is not, funds can be lost on SVM delivery to a zero receiver. Local EVM transfer to zero usually reverts instead of burning.

■ Code

CrossChainERC4626Adapter.sol :

```
function _requireValidEvmBeneficiary(bytes32 beneficiary) private pure {
    if (beneficiary >> 160 != 0) revert InvalidEVMAddress(beneficiary);
    // bytes32(0) still OK here → becomes address(0) on decode
}
```

■ Recommendation

If zero should be rejected: require a non-zero beneficiary for both chain families, keep the high-96-bits packing check on EVM, and also reject a zero `tokenReceiver` on SVM. If zero is intended, document that explicitly.

Suggested fix — CrossChainERC4626Adapter.sol :

```
function _requireValidEvmBeneficiary(bytes32 beneficiary) private pure {
    if (beneficiary == bytes32(0)) revert InvalidRecipient();
    if (beneficiary >> 160 != 0) revert InvalidEVMAddress(beneficiary);
}
```

■ Acceptance criteria

- A negative test that a zero beneficiary is rejected on the EVM path.
- A negative test that a zero `tokenReceiver` is rejected on the SVM path, exercising the branch where the EVM check is currently skipped.

■ Status

Pending

[↑ Back to index](#)

Committed cooldown principal is gated on entry-side switches, with no escape hatch

Field	Value
Severity	Medium
Source	<code>cross-chain-erc4626-vault/src/CrossChainERC4626Adapter.sol</code>
Function	<code>_processPayload</code>
ID	M-03
Prior IDs	CCA-1 (remediation brief); I-01 + I-06 (findings consolidation)
Commit	<code>cde2faffec4470c9a3f33103efd6a52d027cd696</code>
Status	Pending

Consolidation note. This entry merges three prior findings that share a root cause: CCA-1 (*“Committed cooldown principal is trapped by entry-side switches”*), I-01 (*“Coupled ingress/exit pause flags freeze in-flight cooldown assets”*), and I-06 (*“Open cooldowns can only be finished from the original source chain and sender”*). The pause-flag consequence, the source-chain-binding consequence, and the absence of an escape hatch are three faces of the same design property: remote cooldowns are owned by the adapter rather than by the end user. The proof of concept is retained from I-01.

■ Severity reconciliation

The findings consolidation rated this **Informational** and flagged in its own open-items list that **Low** was probably more appropriate. The remediation brief rated it **Medium**. This report adopts **Medium**, on the following basis.

Where the freeze is caused by a *reversible* trigger — `setProcessingEnabled(false, false)` — the loss is bounded to message fees plus delay, which is Low. Where it is caused by an *irreversible* trigger — a permanently retired CCIP lane, or `setChainType(src, NONE)` on a chain that cannot be re-enabled — the principal is unrecoverable, because no hub-side settlement path exists. The severity is set by the irreversible case. This rating rests on an assumption about Chainlink’s lane-retirement notice policy; see §9.

■ Description

The cross-chain adapter gates all four vault actions behind two global flags,

`depositsEnabled` and `redeemsEnabled`. Exit-path actions are bound to the same toggles as ingress-path actions:

Action	Direction	Gated by
DEPOSIT	ingress	<code>depositsEnabled</code>
COOLDOWN (initiate)	ingress	<code>redeemsEnabled</code>
UNSTAKE (claim completed cooldown)	exit	<code>redeemsEnabled</code>
CANCEL_COOLDOWN (abort, remind shares)	exit	<code>depositsEnabled</code>

The fork converts an atomic redemption into a two-message protocol, creating the first class of user value the adapter custodies across transactions. Release of that already-committed principal is gated on the same switches and the same chain allowlist that gate new business.

Because remote cooldowns are owned by the adapter rather than the end user, both flags are necessarily global — there is no per-user pause. Restricting the adapter on `StakedAsset` has the same blast radius. These mappings cannot be split after deploy.

Operators typically disable `redeemsEnabled` to halt new inbound redemptions or cooldowns. Because of the coupling, that same action blocks users who have **already completed** their cooldown from reclaiming assets cross-chain. Disabling `depositsEnabled` similarly prevents cancellation of an in-flight cooldown.

Source-chain binding. `cooldownShares` is called by the adapter, so `StakedAsset` records the adapter as owner and moves assets into the silo. The adapter then stores `cooldownAccounts[sourceChain][id] = bytes32(message.sender) . UNSTAKE` and `CANCEL_COOLDOWN` must therefore arrive as a new CCIP message from that same chain and sender, and `processMessage` requires the chain to still be configured (`onlyValidChain`). This binding *is* the ownership check — it is what prevents one user from claiming another's cooldown — but it means a source chain with open cooldowns cannot be decommissioned without stranding them. A replacement adapter cannot take over: silo cooldowns stay bound to the original adapter address, and cooldown IDs are not reusable.

■ Failure path

A routine `setChainType(src, NONE)` during a lane decommission, or `setProcessingEnabled(false, false)`, terminates all in-flight cooldown settlement. While the condition holds, no action is available to either the user or the operator that settles the position.

The loss is bounded to the messages, not the principal, **for reversible triggers**. The failed `processMessage` self-call rolls back atomically: `cooldowns[adapter][id]` and the AssetSilo balance are untouched, and `cooldownAccounts[src][id]` retains the owner. Once a reversible trigger is reverted, the same users settle with fresh messages, at a cost of the lost message fees plus the delay. An **irreversible** trigger — a permanently retired CCIP lane — is materially different, and is the basis for the Medium rating rather than Low.

■ No escape hatch exists

Every external and public function across all 1,087 lines was enumerated. The file contains four ERC-20 movement sites; the only two that move value out are `:555` (inbound delivery) and `:766` (failed-message local recovery), and both recovery entry points bail on `NoRefundableTokenAmounts` because an UNSTAKE record is tokenless. `recoverNative` moves native only. The fork deleted `withdrawFee`, the legacy contract's only other operator-directed transfer. On the callee side, `StakedAsset.unstake` and `cancelCooldown` key off `cooldowns[msg.sender][id]`, so only the adapter can release its own cooldown.

■ Impact

Routine operational controls (pausing ingress) can unintentionally freeze withdrawal or cancellation for users with in-flight cooldowns. The condition can arise without malice — it is the expected consequence of a normal pause action.

Not a theft path. Under a reversible trigger this is a temporary availability issue with no user-accessible self-rescue. Under an irreversible trigger — lane retirement, or a source chain that cannot be re-enabled — assets remain in the silo with no settlement path at all, and source-chain decommissioning becomes a fund-availability event that must be sequenced accordingly.

■ Code

CrossChainERC4626Adapter.sol :

```
// :587, UNSTAKE branch
if (!redeemsEnabled) revert RedeemsDisabled();
// :601, CANCEL_COOLDOWN branch
if (!depositsEnabled) revert DepositsDisabled();
// plus onlyValidChain on both, against a mutable setChainType allowlist

if (action == Action.DEPOSIT) {
    if (!depositsEnabled) revert DepositsDisabled();
} else if (action == Action.UNSTAKE) {
```

```

        if (!redeemsEnabled) revert RedeemsDisabled();
        // Extract amount to be unstaked with specific cooldown Id
        IStakedAsset.Cooldown memory cooldown = IStakedAsset(vault).cooldowns(address

outputAmount = cooldown.assets;

        IStakedAsset(vault).unstake(address(this), id);
        outputToken = assetToken;
    } else if (action == Action.COOLDOWN) {
        if (!redeemsEnabled) revert RedeemsDisabled();
        // The sent token must be vault minted token
        if (inputToken != vault) revert InvalidActionInputToken();
        (outputAmount, cooldownId) = IStakedAsset(vault).cooldownShares(inputAmount);
    } else if (action == Action.CANCEL_COOLDOWN) {
        if (!depositsEnabled) revert DepositsDisabled();
        outputAmount = IStakedAsset(vault).cancelCooldown(id);
        outputToken = vault;
    }
}

mapping(uint64 chain => mapping(uint256 id => bytes32 account)) public cooldownAc

function processMessage(...) external onlySelf onlyValidChain(message.sourceChain
    if (action == Action.CANCEL_COOLDOWN || action == Action.UNSTAKE) {
        if (cooldownAccounts[message.sourceChainSelector][cooldownId] != bytes32(
            revert InvalidCooldown();
        }
        if (action == Action.COOLDOWN) {
            cooldownAccounts[message.sourceChainSelector][id] = bytes32(message.sende
        }
        // no hub-side unstake / cancel / reassign
    }
}

```

■ Proof of Concept

```

function test_PoC_unstake_trapped_when_redeems_disabled() public {
    // 1. User deposits and starts a cooldown (id 0) from the source chain.
    _depositAndStartCooldown();

    // Sanity: assets are now escrowed in the silo, cooldown recorded for the sen
    assertEq(s_asset.balanceOf(address(s_silo)), 100 ether, "assets should be in
    assertEq(s_adapter.cooldownAccounts(SOURCE_CHAIN_SELECTOR, 0), _toBytes32(ORI

    // 2. Cooldown period elapses: the user is now entitled to withdraw.
    vm.warp(block.timestamp + 7 days);
}

```

```

// 3. Operator disables redeems to stop *new* inbound cooldowns/redemptions.
//     (deposits stay enabled)
s_adapter.setProcessingEnabled(true, false);

// 4. User submits the tokenless UNSTAKE control message.
bytes32 unstakeId = keccak256("unstake-blocked");
Client.Any2EVMMessage memory unstake = _buildMessage(
    unstakeId,
    SOURCE_CHAIN_SELECTOR,
    _ccipEvmSender(ORIGINAL_SENDER),
    _encodePayload(2, 0, _toBytes32(ORIGINAL_SENDER), 100 ether, false),
    address(0),
    0
);
s_router.routeMessage(address(s_adapter), unstake);

// 4a. Processing failed purely because of the flag: message is marked failed
assertEq(
    uint256(uint8(s_adapter.messageErrorCode(unstakeId))),
    uint256(uint8(CrossChainERC4626Adapter.ErrorCode.BASIC)),
    "unstake should fail while redeems disabled"
);

// 4b. Funds are frozen: still in the silo, still nothing delivered to the us
//     and the cooldown is untouched (reverted atomically).
assertEq(s_asset.balanceOf(address(s_silo)), 100 ether, "assets remain stuck");
assertEq(s_asset.balanceOf(ORIGINAL_SENDER), 0, "user received nothing");
(uint256 assets,) = s_vault.cooldowns(address(s_adapter), 0);
assertEq(assets, 100 ether, "cooldown still fully intact");

// 4c. No recovery path: the failed message carries no tokens, so both the
//     cross-chain refund and the local recovery revert. The user cannot self
vm.expectRevert(
    abi.encodeWithSelector(CrossChainERC4626Adapter.NoRefundableTokenAmounts.
);
s_adapter.refundFailedMessage(unstakeId);

vm.prank(LOCAL_REFUND);
vm.expectRevert(
    abi.encodeWithSelector(CrossChainERC4626Adapter.NoRefundableTokenAmounts.
);
s_adapter.recoverFailedMessageLocally(unstakeId);

// 5. Only the operator re-enabling redeems unblocks the user.
s_adapter.setProcessingEnabled(true, true);
bytes32 retryId = keccak256("unstake-retry");
Client.Any2EVMMessage memory retry = _buildMessage(

```

```

        retryId,
        SOURCE_CHAIN_SELECTOR,
        _ccipEvmSender(ORIGINAL_SENDER),
        _encodePayload(2, 0, _toBytes32(ORIGINAL_SENDER), 100 ether, false),
        address(0),
        0
    );
    s_router.routeMessage(address(s_adapter), retry);

    assertEq(
        uint256(uint8(s_adapter.messageErrorCode(retryId))),
        uint256(uint8(CrossChainERC4626Adapter.ErrorCode.NONE)),
        "unstake succeeds once redeems re-enabled"
    );
    assertEq(s_asset.balanceOf(ORIGINAL_SENDER), 100 ether, "user finally receive");
    assertEq(s_asset.balanceOf(address(s_silo)), 0, "silo drained");
}

```

Step 5 is available only because the trigger in this PoC is reversible. Substitute an irreversible `setChainType(src, NONE)` and no step 5 exists.

Step 4a also demonstrates the mechanic described in **L-01** — the failed tokenless `UNSTAKE` becomes a permanent `BASIC` record.

■ Recommendation

1. **Decouple settlement from the new-business switches.** Remove both reverts above. Neither action accepts new user value, both are already authorized by the `cooldownAccounts` check at :528, and the share-to-asset conversion occurred at cooldown-creation time. Where a kill switch is required for these paths, it should be a separate `exitsEnabled` flag that is never cleared while unsettled entries exist.
2. **Exempt UNSTAKE and CANCEL_COOLDOWN from onlyValidChain** where `cooldownAccounts[src][id]` is already populated — that mapping constitutes the authorization — or snapshot the source chain's validity at COOLDOWN time.
3. **Add a bounded, non-custodial local escape hatch.** Record a hub-chain fallback beneficiary alongside `cooldownAccounts[src][id]` at :548 (the payload's beneficiary word is unused on that path) and expose `settleCooldown(uint64 srcSelector, uint256 id)`, calling `unstake(address(this), id)` and paying only that recorded address. A general administrative `rescueTokens` would be a worse trade, permitting the admin to drain every outstanding cooldown; the hatch must be beneficiary-pinned.

If the current flag mapping is retained, document it clearly before deploy so operators understand that pausing deposits also disables cancel-cooldown. These flag mappings cannot be split post-deploy.

Interim operational control. Pending remediation, `setChainType(sel, NONE)` and `setProcessingEnabled(false, false)` must not be invoked while cooldowns are outstanding on the affected lane. Add an operational check that queries outstanding `cooldownAccounts` entries for a chain before decommissioning it. This belongs in the operations runbook irrespective of the code fix.

■ **Acceptance criteria**

- A test that pauses processing with an outstanding cooldown and demonstrates the user still settles.
- A test that calls `setChainType(src, NONE)` with an outstanding cooldown on that lane and demonstrates the user still settles.
- Where the escape hatch is implemented: tests that it pays only the recorded beneficiary, and that it cannot be invoked for an unmatured cooldown.

■ **Status**

Pending

[↑ Back to index](#)

M-04 | **Medium**

Fee-exclusive marking permits externalization of the child’s exit fee

Field	Value
Severity	Medium
Source	tenbin-contracts/src/external/morpho/GenericMorphoAdapter.sol
ID	M-04
Prior IDs	GMA-2a (remediation brief)
Commit	633540ee51d7a069f4a5a53feccd8e3498d1ff14

Status	Pending
--------	---------

■ Description

`realAssets()` values the position using `convertToAssets`, which is fee-exclusive under ERC-4626 semantics. Recoverable value is `previewRedeem`, which is fee-inclusive. Against a child charging a material exit fee, the parent's `totalAssets()` and share price overstate recoverable value by $f \times \text{position}$ for as long as the position is held, and exiting holders are paid out of remaining holders' principal.

■ Failure path

An exiting holder calls the permissionless `forceDeallocate(adapter, maxSharesData, assets, onBehalf)` (`VaultV2.sol:824`). The child burns additional shares to cover its exit fee. Against a compliant child with a 10% exit fee, the position's fee-aware value is 990 assets and the caller's proportional claim is 90, but the parent pays 100.

`forceDeallocatePenalty` defaults to 0 and the protocol caps it at 2%, so a larger child exit fee cannot be neutralized by the penalty.

■ The transient cache is an amplifier, not the mechanism

This finding may be misread as a `multicall` / `firstTotalAssets` caching problem, leading to the conclusion that correcting the cache resolves it. It does not. With batching removed entirely and `forceDeallocate` and `redeem` executed as two separate transactions, the first accrual occurs inside `withdraw`, the loss realizes immediately, and `previewRedeem(100e18)` still pays approximately 98.99e18 against a 90e18 fee-aware claim. The cache accounts for approximately 1.01 of the approximately 10 units of excess.

■ The Gate does not bound this finding

`Deploy.s.sol:253-261` installs `Gate.sol` on all four gate slots, returning true only for the CollateralManager. This does not bound severity, for four independent reasons. The first applies to the deployed configuration:

1. `forceDeallocate` is permissionless even with all four gate slots installed.

Demonstrated: with the production `Gate` installed and `canReceiveAssets(attacker) == false`, that same ungated attacker calls `forceDeallocate(adapter, abi.encode(type(uint256).max), 100e18, manager)` successfully — allocation goes to zero, assets land in the parent's idle balance, and at the default zero penalty the victim's shares are untouched. The gates constrain who may hold shares and move assets into or out of the vault; they do not constrain who may move assets between an adapter and

idle.

2. `GenericMorphoAdapterFactory.sol:28` is permissionless and accepts an arbitrary `parentVault`. Nothing constrains the parent to a gated Tenbin vault.
3. The repository's own fork tests construct ungated multi-holder parents (`GenericMorphoAdapterFactoryForkTest.t.sol:141`, `:176`).
4. Even with a single holder, the inflated NAV propagates into Tenbin's collateral accounting at `CollateralManager.sol:452-453`.

■ Impact

Exiting holders are paid above their fee-aware claim, out of remaining holders' principal, whenever the child charges a material exit fee. The inflated NAV additionally propagates into Tenbin's collateral accounting.

Reachability against the two children named in the deploy configuration is unconfirmed — see determination 3 in §10 — but the Medium rating does not depend on it, because the factory is permissionless and the parent is unconstrained.

■ Recommendation

Value the complete child position with

`vault.previewRedeem(vault.balanceOf(address(this)))`, matching the pinned Morpho adapter and ERC-4626 fee semantics, and use fee-aware previews for any share-bound validation.

The shared function must not be wrapped naively. See M-05, which resolves both findings with a single change and documents the constraint.

Independently of the code change: targets with material or nonlinear entry or exit losses should be rejected unless invariant tests establish, across supported sizes, that each allocation increases fee-aware position NAV by approximately the assets transferred and each deallocation reduces it by approximately the assets returned.

■ Acceptance criteria

- `realAssets()` returns a fee-inclusive value, and the invariant `realAssets() == maxWithdraw(address(this))` holds.
- A test against a child with a 10% exit fee showing the parent pays the fee-aware claim, executed both batched and as two separate transactions.

■ Status

Pending

[↑ Back to index](#)

M-05 | Medium

Donated shares count at zero allocation and can never be permissionlessly removed

Field	Value
Severity	Medium
Source	tenbin-contracts/src/external/morpho/GenericMorphoAdapter.sol
ID	M-05
Prior IDs	GMA-2b (remediation brief)
Commit	633540ee51d7a069f4a5a53feccd8e3498d1ff14
Status	Pending

■ Description

Upstream short-circuits `realAssets()` to zero where the parent holds no allocation. The fork removes that guard. Three properties compose into the defect:

- The adapter can be funded by anyone at zero allocation** — via `vault.deposit(x, adapterAddress)` or a direct transfer of child shares. Neither is access-controlled; the adapter is an address holding ERC-4626 shares.
- VaultV2 counts them unconditionally.** `VaultV2.sol:657-659` sums `realAssets()` across every registered adapter, with no zero-allocation filter and no try/catch. VaultV2's own adapter specification at `:72` states that *"The totalAssets() calculation ignores markets for which the vault has no allocation"* — it states the requirement without enforcing it, placing the obligation on the adapter.
- The only permissionless exit valve refuses to unwind them.**
`require(_caps.allocation > 0, ZeroAllocation())` at `VaultV2.sol:608` executes

before the `:609` update, so a positive change originating from a donation-only position can never register, and `forceDeallocate` reverts. The file's only two `.allocation` writes are `:581` and `:609`; no other path exists.

■ Upstream provenance

The removed guard exists because an auditor identified this defect and it was corrected.

`lib/vault-v2/audits/2025-09-15-chainsecurity.pdf` §6.3 records it as **CS-MORPHO-VLT2-020**, *"Unallocated Vault Adapter Can Report Assets"*, rated **Design Medium, Code Corrected**; the recorded correction is precisely the guard this fork removed. The enforcing regression, `testSharesDonationResistanceIfNoAllocation`, also ships in the dependency tree.

■ Recovery is not the immediate allocator action it appears to be

`allocate(0)` does register the position, but `allocateInternal` writes the allocation and then validates the caps against the new value. Recovery therefore requires `absoluteCap ≥ donation` **and** either `relativeCap == WAD` or donation-sized relative headroom. Three states defeat it, all timelocked to repair:

- **Between the timelocked `addAdapter` and the timelocked `increaseAbsoluteCap`**, `absoluteCap == 0` and `allocate(0)` reverts `ZeroAbsoluteCap`. This is the precise window the finding requires, and it is two days under the deployed configuration.
- **A donation sized above the adapter's `absoluteCap`** reverts `AbsoluteCapExceeded`, forcing a further timelocked increase, which itself widens the cap for real allocations. The deploy script's `initial_absolute_cap` of 100m USDC is granted to the **upstream** `MorphoVaultV1Adapter` (`Deploy.s.sol:206`, `:228`); a Generic adapter added subsequently carries a distinct `adapterId` whose cap begins at zero.
- **A fractional relative cap** reverts `RelativeCapExceeded` even where the absolute cap is ample.

Cap operations may already be pending or be submitted concurrently, so recovery does not invariably incur a fresh full delay.

■ Absorption is throttled, at the weakest permitted setting

`VaultV2.sol:661` caps growth via `maxRate`, so absorbing a donation worth 100% of NAV requires approximately half a year at the protocol maximum. All seven vault configs set `max_rate = 63419583967`, which is exactly `MAX_MAX_RATE`. See observation **N-8**: the adjacent comment misstates this value by a factor of ten.

■ Secondary effect on unrelated adapters

A stuck donation raises the relative-cap denominator for every other adapter in the same parent: a 100-asset donation into one adapter's target eventually doubles a second, unrelated adapter's 50% relative cap in absolute units. This operates through accrued `totalAssets` and is therefore rate-limited by the same `maxRate`; it cannot be triggered instantly.

■ Impact

A free permissionless grief rather than a profitable attack — the donation is forfeited in full, with no profit to the donor. The cost to the protocol is distorted share pricing and withdrawal accounting, and a position the only permissionless exit valve refuses to touch, until a timelocked intervention completes.

■ Recommendation — note the constraint before implementing

The shared function must not be wrapped naively. The fork reuses `realAssets()` internally at :78 and :105 to compute `newAllocation`, whereas upstream inlines the unguarded expression. Guarding the shared function would cause the first `allocate()` from a zero-allocation state to compute `change = 0`, leaving `allocation == 0` while real assets sit in the target — after which `VaultV2.sol:608` permanently blocks deallocation. **That converts this finding into a fund lock.** Split the reads:

```
function _position() internal view returns (uint256) {
    return vault.previewRedeem(vault.balanceOf(address(this)));
}

function allocation() public view returns (uint256) {
    return parentVault.allocation(adapterId);
}

function realAssets() public view returns (uint256) {
    return allocation() != 0 ? _position() : 0;
}

// in allocate (:78) and deallocate (:105):
uint256 newAllocation = _position();    // NOT realAssets()
```

This single change resolves both M-04 and M-05 and restores the invariant `realAssets() == maxWithdraw(address(this))`.

■ Acceptance criteria

- `testSharesDonationResistanceIfNoAllocation` is ported from the dependency tree and passes.
- A test covering the zero-to-positive allocation transition, establishing that the first `allocate()` following a donation registers a `nonzero change` and that the position is subsequently deallocatable. This is the regression that detects the fund-lock condition above.
- `realAssets()` returns 0 at zero allocation with a donated balance present.

■ Status

Pending

[↑ Back to index](#)

■ Low

L-01 | ■ Low

Failed `UNSTAKE` / `CANCEL_COOLDOWN` messages stay permanently unresolved

Field	Value
Severity	Low
Source	<code>cross-chain-erc4626-vault/src/CrossChainERC4626Adapter.sol</code>
ID	L-01
Prior IDs	L-01 (findings consolidation), N1 (remediation brief)
Commit	<code>cde2faffec4470c9a3f33103efd6a52d027cd696</code>
Status	Pending

■ Description

`UNSTAKE` and `CANCEL_COOLDOWN` must arrive with empty `destTokenAmounts` . If `processMessage` reverts, vault state is unchanged and the user can retry with a new CCIP

message — so no principal is at risk.

However, `ccipReceive` still marks that `messageId` as `BASIC`. Both `refundFailedMessage` and `recoverFailedMessageLocally` require a positive token amount, so both revert `NoRefundableTokenAmounts`. Nothing can move a tokenless `BASIC` record to `RESOLVED`.

A failed `UNSTAKE` or `CANCEL_COOLDOWN` therefore leaves a permanent `BASIC` record. No principal is locked; only the failed-message bookkeeping cannot be cleared.

The user-paid message is also burned. `ccipReceive` (:482-495) deliberately never reverts, so the Router returns success and the OffRamp records `SUCCESS`, rendering CCIP manual re-execution unavailable. Combined with the bare `catch`, a failed message is both unrecoverable and undiagnosable — the retry is a *new* message at a *new* fee, not a re-execution of the paid one. This depends on an assumption about OffRamp re-execution eligibility recorded in §9.

Example. A remote `UNSTAKE` submitted before the cooldown ends → `processMessage` reverts → silo untouched → that `messageId` stays `BASIC` forever. A later, correctly-timed `UNSTAKE` still succeeds normally, at the cost of a fresh message fee.

■ Impact

Operations and monitoring noise, plus a burned message fee per failure. Over time the failed-message set accumulates entries that can never be cleared, degrading the signal value of that set for incident response.

Distinct from H-01, which concerns token-carrying failures with genuinely stranded ERC-20s.

■ Code

`CrossChainERC4626Adapter.sol` :

```
function ccipReceive(Client.Any2EVMMessage calldata message) external onlyRouter
    try this.processMessage(message) {
        emit MessageSucceeded(message.messageId);
    } catch {
        // ANY failure, including tokenless UNSTAKE/CANCEL, becomes BASIC
        messageErrorCode[message.messageId] = ErrorCode.BASIC;
        _storeFailedMessageRecord(message.messageId, message);
        emit MessageFailed(message.messageId);
    }
}
```

```

if (action == Action.CANCEL_COOLDOWN || action == Action.UNSTAKE) {
    if (message.destTokenAmounts.length != 0) revert InvalidTokenCount(...);
}

function refundFailedMessage(bytes32 messageId) external payable nonReentrant {
    if (!_anyPositiveDestAmount(record.destTokenAmounts))
        revert NoRefundableTokenAmounts(); // tokenless BASIC cannot reach RESOLV
}

```

■ Recommendation

Either do not record `BASIC` for tokenless `UNSTAKE` / `CANCEL_COOLDOWN` failures, or add a resolve path that clears `BASIC` when there is nothing to refund.

Independently, bind the error data in the `catch` rather than discarding it. The bare `catch` is what makes a failed message undiagnosable, and it also prevents the negative tests in I-04 from attributing a revert to the gate that produced it.

Suggested fix — Option A (allow clearing tokenless `BASIC`):

```

function resolveFailedMessage(bytes32 messageId) external {
    if (messageErrorCode[messageId] != ErrorCode.BASIC) revert MessageNotFailed(m
FailedMessageRecord memory r = failedMessageRecords[messageId];
    if (_anyPositiveDestAmount(r.destTokenAmounts)) revert NoRefundableTokenAmoun
// admin or original sender
    messageErrorCode[messageId] = ErrorCode.RESOLVED;
    delete failedMessageRecords[messageId];
    delete refundChainFamilySnapshot[messageId];
}

```

Suggested fix — Option B (do not mark tokenless failures as `BASIC`):

```

function ccipReceive(Client.Any2EVMMessage calldata message) external onlyRouter
    try this.processMessage(message) {
        emit MessageSucceeded(message.messageId);
    } catch (bytes memory err) {
        if (message.destTokenAmounts.length > 0) {
            messageErrorCode[message.messageId] = ErrorCode.BASIC;
            _storeFailedMessageRecord(message.messageId, message);
        }
        emit MessageFailed(message.messageId, err);
    }
}

```

■ Acceptance criteria

- A test that a failed tokenless `UNSTAKE` can be moved to `RESOLVED` , or is never recorded as `BASIC` .
- A test asserting the emitted failure carries attributable error data.

■ Status

Pending

[↑ Back to index](#)

L-02 |  Low

Solana path always assumes 9 decimals (USDC is 6)

Field	Value
Severity	Low
Source	<code>cross-chain-erc4626-vault/frontend/client/src/components/vault/SendDialog.tsx</code>
ID	L-02
Prior IDs	L-02 (findings consolidation)
Commit	— <i>(populate before delivery)</i>
Status	Pending

■ Description

Before sending, the frontend converts the user's typed amount into a raw integer using `sourceDecimals` . For Solana sources, `SendDialog` falls back to `decimals = 9` , and `readSourceToken` in `send.ts` **also** returns `{ decimals: 9 }` for any non-EVM source — so neither path rescues the case.

That assumption is wrong for many tokens. USDC on Solana has 6 decimals.

Example. A user types `1` intending 1 USDC:

Assumed decimals	Raw amount	Actually bridged
6 (correct)	1_000_000	1 USDC ✓
9 (current)	1_000_000_000	1000 USDC ✗

■ Impact

Users can bridge far more than the amount they typed on Solana — a 1000× overshoot in the USDC case. The funds are not lost (they arrive at the intended beneficiary), but the user commits vastly more capital than intended, and the transaction may fail outright on insufficient balance.

This is frontend-only; a corrected client fixes it with no contract change.

■ Code

`SendDialog.tsx`:

```
const sourceDecimals =
  sourceTokenMeta?.decimals
  ?? (source?.family === "SVM" ? 9 : deliveredDecimals);

const amountRaw = parseAmount(amount, sourceDecimals);
// amountRaw is what gets bridged

// Example — USDC on Solana has 6 decimals, not 9:
// parseAmount("1", 6) = 1_000_000      ✓ 1 USDC
// parseAmount("1", 9) = 1_000_000_000  ✗ 1000 USDC
```

■ Recommendation

Read the token's real decimals for Solana via SPL token metadata. If unknown, fall back to `deliveredDecimals` as the EVM path does — do not hardcode `9`. Apply the same correction to `readSourceToken` in `send.ts`, since it carries the identical fallback.

Consider adding a confirmation step that displays the resolved raw amount before signing.

■ Acceptance criteria

- A test that a 6-decimal SPL token resolves to 6 decimals through both `SendDialog` and `readSourceToken`.

- No hardcoded decimal fallback remains on the SVM path.

■ Status

Pending

[↑ Back to index](#)

L-03 |  Low

A single static `liquidityData` blob cannot serve both allocation directions

Field	Value
Severity	Low
Source	<code>tenbin-contracts/src/external/morpho/GenericMorphoAdapter.sol</code>
ID	L-03
Prior IDs	GMA-1 (remediation brief), M-03 (findings consolidation)
Commit	<code>633540ee51d7a069f4a5a53feccd8e3498d1ff14</code>
Status	Pending — see determination 4 in §10

■ Severity reconciliation

The findings consolidation rated this **Medium**, on the basis that `Deploy.s.sol` already wires the *official* V1 adapter through `setLiquidityAdapterAndData`, making the hazardous pattern easy to replicate. The remediation brief rated it **Low**, on the basis that two escape hatches were confirmed and that `GenericMorphoAdapter` appears in neither `script/` nor `config/`.

This report adopts **Low**. The escape hatches are specific and verified:

`setLiquidityAdapterAndData` (:618) is allocator-gated and **not** timelocked, and `forceDeallocate` (:824) is permissionless with caller-supplied `data`. A misconfiguration is therefore repairable within a single allocator transaction, and individual users retain a self-service exit in the interim. The Medium rationale identifies a real ease-of-error concern, which is preserved as the deployment-hazard framing below, but ease of error without an irreversible consequence is a Low.

If determination 4 in §10 confirms that `GenericMorphoAdapter` **is** intended for liquidity-adapter use, this should be re-rated to Medium and remediated in full rather than documented.

■ Description

`GenericMorphoAdapter` encodes slippage in `data` as a single `uint256` carrying **opposite** roles depending on direction:

Call	<code>data</code> meaning	Check
<code>allocate</code>	<code>minShares</code>	reject if too few shares minted
<code>deallocate</code>	<code>maxShares</code>	reject if too many shares burned

These bounds are inherently **trade-specific** — they depend on trade size and the current convert rate. That design fits **manual** allocator calls (`VaultV2.allocate` / `deallocate` with fresh `data` supplied each time), which is what the fork tests exercise.

Morpho Vault V2's **liquidity adapter** works differently: VaultV2 stores a single `bytes liquidityData` (`:235`) and supplies the same blob to both directions (`:775` , `:800-801`), for both automatic `allocate` (on `enter`) and automatic `deallocate` (on `exit` when idle is short). The two interpretations are inverted: a value strict enough to protect deposits is either too permissive or fatal for withdrawals, and the converse. No single value protects both.

Tenbin's `CollateralManager` only calls `deposit` / `withdraw` on the parent Vault V2. When a liquidity adapter is set, the vault itself performs the `allocate/deallocate` calls. `Deploy.s.sol` wires the official V1 adapter exactly this way:

```
setLiquidityAdapterAndData(adapter, new bytes(0));
```

The official `MorphoVaultV1Adapter` requires empty `data` and performs no min/max checks, so a single empty blob satisfies both directions. With Generic's single- `uint256` encoding carrying opposite min/max semantics, **no fixed `liquidityData` value can satisfy both auto-allocate and auto-deallocate** when `assets > 0` .

■ Impact

- The manual per-transaction `data` path is unaffected and works correctly.
- Under the current encoding, `GenericMorphoAdapter` **cannot** be used as a Vault V2

liquidity adapter. Wiring it in the same shape as Deploy's V1 liquidity setup causes `CollateralManager deposit / withdraw` to fail whenever the parent vault auto-allocates or auto-deallocates.

- Where the adapter is wired as `liquidityAdapter`, either all deposits revert, or — under the worse ordering — deposits succeed and all liquidity-backed withdrawals revert until an allocator repairs the configuration or each user pays gas to `forceDeallocate`.

This is a deployment-configuration hazard rather than a fund-loss path: the failure is a revert, not a silent mispricing. It is nonetheless an easy mistake to replicate, because the wiring pattern that triggers it is the one already present in `Deploy.s.sol` for the official adapter.

■ Recommendation

Make `data` direction-agnostic so a single stored blob serves both calls:

```
uint256 minShares = data.length == 0 ? 0 : abi.decode(data, (uint256))
uint256 maxShares = data.length == 0 ? type(uint256).max : abi.decode(data, (uint256))
```

Alternatively, encode both bounds in one struct — `abi.decode(data, (uint256 minSharesOnAllocate, uint256 maxSharesOnDeallocate))` — and use only the relevant field per direction, allowing a single frozen blob to express a permissive bound in each direction (`minShares = 0, maxShares = type(uint256).max`) for liquidity-adapter use while still supporting tight bounds on manual calls. In either case, move the decode below the `assets > 0` guard.

Accepting empty `data` as an explicit “no slippage check” sentinel, matching `MorphoVaultV1Adapter` behaviour, is an acceptable alternative.

Where the encoding is not changed, explicit `NatSpec` must state that this adapter **MUST NOT** be set as `liquidityAdapter`, and the factory or deploy script should reject or warn on `setLiquidityAdapterAndData` with a `GenericMorphoAdapter` target. Documentation alone leaves a live footgun and is the least preferred option.

Note that these bounds are allocator-convenience guards, not security controls:

`forceDeallocate` is permissionless and accepts caller-chosen `data`, so any caller can already supply `maxShares = type(uint256).max`.

■ Acceptance criteria

- A test wiring the adapter through `setLiquidityAdapterAndData` with a single blob and exercising both a deposit and a withdrawal through it.

■ Status

Pending

[↑ Back to index](#)

L-04 |  Low

Permissionless dust returns can exhaust the shared native CCIP reserve

Field	Value
Severity	Low
Source	cross-chain-erc4626-vault/src/CrossChainERC4626Adapter.sol
ID	L-04
Prior IDs	CCA-3 (remediation brief)
Commit	cde2faffec4470c9a3f33103efd6a52d027cd696
Status	Pending

■ Description

Outbound CCIP messages are paid from a shared native balance held by the contract.

`sendToken` validates only that the balance covers the current quote:

```
// :837-838
uint256 fee = IRouterClient(ROUTER).getFee(destinationChainSelector, message);
if (address(this).balance < fee) revert InsufficientNativeBalance(fee, address(th
```

Any remote account may initiate minimum-size returns, each consuming a full outbound fee, driving the reserve below the next quote and stalling outbound messaging for all users.

■ Intersection with M-03

A cooldown initiated while the reserve is healthy commits the system to a later tokenless exit message. Where the reserve is exhausted by that point, the exit cannot be paid — the same trapped-principal condition described in M-03, reached economically rather than through operator action.

■ Impact

Denial of outbound messaging for all users until the reserve is replenished, at an attacker cost of the dust plus their own source-side fees. Combined with M-03, an economically-induced freeze of committed cooldown principal.

■ Recommendation

- 1. Require the user to prepay the outbound native quote, or charge a lane- and token-specific asset fee conservatively sufficient to replenish it. Prepaid per-user credits with explicit global and per-user budgets are an alternative.
- 2. Reject uneconomic dust returns using lane-specific minimum output amounts. Rate limiting alone does not restore solvency.
- 3. Where cooldown initiation commits the system to a later tokenless exit, reserve or escrow sufficient credit for that exit. This item closes the M-03 intersection.
- 4. Expose reserve sufficiency and expected outbound cost to clients before cooldown initiation, and monitor the balance against the current quote.

■ Acceptance criteria

- A test that depletes the reserve through repeated minimum-size returns and demonstrates outbound messaging still functions for a committed cooldown.

■ Status

Pending

[↑ Back to index](#)

L-05 |  Low

Inflation-resistance requirement removed while the target set was widened

Field	Value
Severity	Low
Source	tenbin-contracts/src/external/morpho/GenericMorphoAdapter.sol
ID	L-05

Prior IDs	GMA-3 (remediation brief)
Commit	633540ee51d7a069f4a5a53feccd8e3498d1ff14
Status	Pending

■ Description

Upstream carries a deployment requirement the fork deleted: *“This adapter must be used with Morpho Vaults V1 that are protected against inflation attacks with an initial deposit.”*

The fork simultaneously widened the target set from MetaMorpho (virtual shares, decimal offset) to “any standard ERC4626”, which includes solmate-style vaults without virtual shares. `allocate` carries no `shares != 0` invariant; the caller-supplied `minShares` is the only protection.

```
if (assets > 0) {
    shares = vault.deposit(assets, address(this)); // :74
    if (shares < minShares) revert InsufficientShares(); // :75
}
```

■ Impact scales with attacker capital

Let s be the attacker’s seed in the child’s smallest share unit and x the victim’s parent deposit. The victim mints exactly one share while $A \leq Xs < 2A$, so the attacker minimises A just above $Xs/2$ and nets $P \approx Xs / (2(s+1))$, which tends to $X/2$:

seed s	temporary capital	attacker profit	victim loss	attacker ROI
1	$\approx 0.5 \cdot X$	$\approx 0.25 \cdot X$	$\approx 25\%$	$\approx 50\%$
10	$\approx 5 \cdot X$	$\approx 0.45 \cdot X$	$\approx 45\%$	$\approx 9\%$
100	$\approx 50 \cdot X$	$\approx 0.50 \cdot X$	$\approx 50\%$	$\approx 1\%$

$s = 1$ is the economically dominant attack; the larger rows establish the severity ceiling. Against a child minting zero shares for the victim — permitted by EIP-4626 and not excluded by “standard” — the attacker remains sole shareholder and the loss approaches 100%.

■ The capital requirement is not a mitigation

In the ordinary third-party sandwich the donation must straddle the victim's transaction, requiring the attacker to hold capital across two transactions. Where the attacker is also the depositor, however, the attack composes within a single transaction: flash-borrow, seed 10 wei, donate approximately 500 assets, self-deposit 100 through a weak-bound automatic adapter, redeem the child position, then redeem the parent position out of idle at VaultV2's transient cached price before the drain is recognised, repay principal plus a 5 bps fee, and retain more than 45 assets. The loss falls on existing parent holders. This variant additionally requires an ungated parent and idle parent liquidity; the deployed `Gate` blocks the self-deposit, and `GenericMorphoAdapter` is absent from `script/` and `config/`.

■ The explicit allocator path is unaffected

`parentVault.allocate(adapter, abi.encode(previewDeposit(100e18)), 100e18)` reverts any such sandwich via `minShares`. The fork is stronger than upstream on this path, which carries no slippage guard at all. Exposure is confined to the removed deployment requirement together with the unbounded liquidity path (L-03).

There is no impact against any target configured or tested in this repository — sUSDS, Aave stataUSDC, MockERC4626 and MetaMorpho are all deeply seeded or virtual-share-protected.

■ Recommendation

Target admission is the only complete fix. Restrict the adapter to children with virtual-share or otherwise inflation-resistant accounting, or to children carrying a provably adequate seed the attacker does not own. No enforceable check inside `allocate` closes the class, because the attacker controls the child's exchange rate at the moment the adapter reads it. Restore the upstream `NatSpec` requirement at :12-16.

Two measures are worth adopting but are **partial**, and are recorded as such because they present as sufficient:

- `if (assets > 0 && shares == 0) revert` blocks only the zero-share variant. The dominant attack mints exactly one share and passes through unaffected. It is free and closes the worst tail, and should be retained, but it is not a fix.
- A constructor `vault.totalSupply() != 0` check is bypassed by pre-seeding: the attacker seeds the child before the deployment transaction and the check passes.

Any `minShares` relied upon must derive from a rate observed before the attacker's donation, never from a `previewDeposit()` taken within the same transaction, which merely blesses the manipulated rate.

■ Acceptance criteria

- NatSpec restored, and target admission enforced by allowlist or by a documented curator checklist.
- Where the `shares == 0` guard is added, its test asserts that it is a partial mitigation — that is, that a one-share sandwich still succeeds — so that it is not subsequently mistaken for the fix.

■ Status

Pending

[↑ Back to index](#)

L-06 |  **Low**

Staked-asset redeem passes validation but can never execute

Field	Value
Severity	Low
Source	<code>tenbin-contracts/src/.../BebopHook.sol</code>
ID	L-06
Prior IDs	BH-1 (remediation brief)
Commit	<code>633540ee51d7a069f4a5a53feccd8e3498d1ff14</code>
Status	Pending

■ Description

A staked-asset redemption passes `verifyHookData` and then reverts unconditionally during execution: only one of the two required allowances is granted, and the pulled input is sized in assets where the flow requires shares. The failure surfaces deep inside `AssetToken._spendAllowance` rather than as a clear rejection at validation.

■ Impact

No fund loss — the transaction reverts. The cost is a wasted signed order and a failure mode that is expensive to diagnose, because the revert originates several call frames below the validation that should have rejected the shape. No `BebopHook` test references the staked asset, so this is not currently caught by the suite (observation **N-7**).

■ Recommendation

Preferred, and consistent with the tested surface — reject the shape at validation. In the redeem branch of `verifyHookData (:233-236)`:

```
} else {
    if (order.order_token != data.inputToken || data.inputToken != controller.asset
        if (order.collateral_token != data.outputToken) revert InvalidOutputToken();
    }
```

Where staked-asset redemption through the hook is intended, both changes are required together:

```
// before controller.redeem, when order.order_token is the staked asset:
IERC20(controller.asset()).forceApprove(address(controller), issuerData.order.asset
// and size the pulled input in SHARES, not assets:
orderInputAmount = IStakedAsset(stakedAsset).previewWithdraw(issuerData.order.asset
```

Both allowances must be zeroed after `controller.redeem` returns. This path additionally requires `stakedAsset()` to be exposed on `IController`, which currently declares only `asset()`.

■ Acceptance criteria

- A test submitting a staked-asset redeem order that either reverts with a clear typed error at validation, or completes end-to-end with both allowances zeroed afterward.

■ Status

Pending

[↑ Back to index](#)

L-07 |  **Low**

Caller-chosen owner defeats the write-once registry guarantee

Field	Value
Severity	Low
Source	tenbin- contracts/src/external/morpho/GenericMorphoAdapterFactory.sol
ID	L-07
Prior IDs	GMAF-1 (remediation brief), I-07 (findings consolidation)
Commit	633540ee51d7a069f4a5a53feccd8e3498d1ff14
Status	Pending — see determination 6 in §10

■ Severity reconciliation

The findings consolidation rated this **Informational**, reasoning that no funds are at risk and the impact is stale references and operator confusion. The remediation brief rated it **Low**, noting additionally that the NatSpec advertises a write-once guarantee the implementation does not provide, and that both obvious fixes are unsafe. This report adopts **Low**: a documented invariant that does not hold is a defect in its own right, independent of current consumer behaviour.

■ Description

`createGenericMorphoAdapter(parent, vault, owner)` uses `CREATE2` with salt `0`. The canonical registry is keyed on the `(parentVault, vault)` pair only, while `owner` is a caller-supplied constructor argument:

```
genericMorphoAdapter[parent][vault]
```

Two deployments for the same pair with different owners produce different `CREATE2` addresses, both succeed — there is no collision at the EVM level — and the second silently **overwrites** the canonical mapping entry.

The key asymmetry is that the deployment address is a function of three inputs while the registry key is a function of two. Any uniqueness guarantee a consumer infers from the registry is therefore weaker than it appears, and weaker than the NatSpec states.

■ Impact

No direct fund risk: the previously deployed adapter continues to exist at its own address and retains whatever authorisation the parent vault granted it. Nothing is destroyed.

The risk is one of stale references and operator confusion:

- Any consumer that resolves an adapter through `genericMorphoAdapter[parent][vault]` will silently begin pointing at the new deployment.
- The original adapter may still hold positions or retain parent-vault authorisation while no longer being discoverable through the registry.
- Off-chain tooling, monitoring, and deploy scripts that treat the registry as canonical will disagree with on-chain reality.

Whether this matters depends on how much authority the registry carries in the surrounding system. If it is used only as a convenience lookup, the impact is minimal; if any privileged flow resolves adapters through it, this should be re-rated.

■ Recommendation

Preferred, and the only option that is safe in isolation: remove the `owner` parameter and derive rescue authority on-chain from `IVaultV2(parentVault).owner()`, as upstream does. The mapping key then covers the entire constructor argument set, a repeat deployment for a pair lands on an occupied `CREATE2` address, `new` reverts, and the write-once guarantee is restored by construction.

Where `owner` must remain a parameter, the mapping cannot remain a bare `(parentVault, vault)` scalar. Key it on all three constructor arguments — `genericMorphoAdapter[parent][vault][owner]`, matching the address derivation exactly — or maintain an append-only list per pair, or require parent or governance authorization before a deployment may claim the canonical entry. Whichever is chosen, emit `AdapterReplaced(parent, vault, oldAdapter, newAdapter)` on overwrite so monitoring can detect it.

Two remediations that present as sufficient and are not:

- A permissionless write-once check, while `owner` remains caller-chosen, converts an overwrite defect into a squatting denial of service. An attacker deploys first with an attacker-chosen `owner`, permanently claims the canonical slot, and the legitimate deployment can never register. Write-once is safe only in combination with removing `owner`, at which point the squatter's deployment is byte-identical to the legitimate one and the grief is benign.
- Changing the `CREATE2` salt is inert. Constructor arguments are already part of the initcode preimage, so varying the salt does not change what the mapping fails to

distinguish.

■ Acceptance criteria

- A test establishing that two deployments for the same `(parentVault, vault)` pair cannot both claim the canonical mapping entry.

■ Status

Pending

[↑ Back to index](#)

Informational

I-01 | Info

Gate manager can be `address(0)` , which locks the vault

Field	Value
Severity	Info
Source	<code>tenbin-contracts/src/external/morpho/Gate.sol</code>
ID	I-01
Prior IDs	I-02 (findings consolidation)
Commit	— <i>(populate before delivery)</i>
Status	Pending

■ Description

The Morpho `Gate` only allows the `manager` address to deposit and withdraw. If `manager` is `address(0)` , no real account matches, so gated vault flows are locked.

Two ways to reach that state:

1. The `manager` storage variable defaults to `address(0)` — the vault is locked from

deployment until the owner sets a real value.

2. `setManager` also permits setting it back to zero, with no guard.

Example. After deploy, `manager` is still the default `address(0)` until the owner sets a real one — or the owner sets it to `address(0)` by mistake. Deposits and withdrawals through the gate all fail until a non-zero manager is set.

■ Impact

Operational lock / misconfiguration. Not a theft path, and recoverable by the owner calling `setManager` with a valid address. Rated Informational because recovery is straightforward and the owner retains control throughout.

Note that the gate's scope is narrower than it appears — see M-04, where `forceDeallocate` is shown to remain permissionless with all four gate slots installed.

■ Code

Gate.sol :

```
address manager; // default = address(0) → vault locked until set

function setManager(address newManager) external onlyOwner {
    manager = newManager; // zero allowed
}

function canReceiveAssets(address account) external view returns (bool) {
    return account == manager; // never true while manager is 0
}
```

■ Recommendation

If locking the vault is not intended, revert when `setManager` is called with `address(0)` . Consider also setting a manager in the constructor so the vault is never deployed in the locked state.

Suggested fix — Gate.sol :

```
function setManager(address newManager) external onlyOwner {
    if (newManager == address(0)) revert ZeroAddress();
    manager = newManager;
}
```

■ Acceptance criteria

- A negative test that `setManager(address(0))` reverts.
- The deployment path establishes a non-zero manager in the same transaction as, or before, the gate becomes reachable.

■ Status

Pending

[↑ Back to index](#)

I-02 | Info

Unused fee roles and outdated docs left after the Tenbin fork

Field	Value
Severity	Info
Source	cross-chain-erc4626-vault/src/CrossChainERC4626Adapter.sol
ID	I-02
Prior IDs	I-03 (findings consolidation), N5 (remediation brief)
Commit	cde2faffec4470c9a3f33103efd6a52d027cd696
Status	Pending

■ Description

Flat fees were removed in the Tenbin fork, but `FEE_SETTER_ROLE` and `FEE_COLLECTOR_ROLE` (:54-55) survive as grantable public constants and are never used. Some documentation still describes the old fee and Legacy behaviour, so it no longer matches the code.

■ Impact

No fund risk. The concern is operational: an operator inspecting on-chain roles may grant `FEE_SETTER_ROLE` believing it controls something, or follow stale documentation describing fee flows that no longer exist. Dead role constants also add noise to any access-control

review.

This compounds with M-01, where an unchanged `typeAndVersion` string makes Tenbin and Legacy adapters indistinguishable — both are cases of fork residue that misleads integrators.

■ Code

CrossChainERC4626Adapter.sol :

```
// Still declared, but fees were removed — these roles are unused
bytes32 public constant FEE_SETTER_ROLE = keccak256("FEE_SETTER_ROLE");
bytes32 public constant FEE_COLLECTOR_ROLE = keccak256("FEE_COLLECTOR_ROLE");
```

■ Recommendation

Remove the unused `FEE_*` roles, or mark them explicitly deprecated with a comment. Update documentation to match Tenbin behaviour.

■ Acceptance criteria

- No unused role constant remains grantable, or each carries an explicit deprecation comment.
- Documentation describing fee flows is removed or corrected.

■ Status

Pending

[↑ Back to index](#)

I-03 | Info

UI shows Success when the vault action actually failed

Field	Value
Severity	Info
Source	cross-chain-erc4626-vault/src/CrossChainERC4626Adapter.sol , cross-chain-erc4626-

	vault/frontend/client/src/components/vault/TrackedMessages.tsx
ID	I-03
Prior IDs	I-04 (findings consolidation)
Commit	— (<i>populate before delivery</i>)
Status	Pending

■ Description

The adapter catches vault-processing failures so the CCIP router call still succeeds. CCIP therefore reports `SUCCESS` even though the adapter emitted `MessageFailed` and set status `BASIC`. The monitor treats CCIP `SUCCESS` as a green success and never checks the adapter's error status.

"Bridge delivered" is not the same as "vault action worked." A failed message with stuck tokens can look successful in the UI.

■ Impact

Users may briefly believe a failed vault action succeeded. On-chain state (`MessageFailed` / `messageErrorCode`) remains authoritative, and the monitor still tracks failed-message records, so this is a display and operational-visibility issue rather than a protocol fund bug.

The practical significance rises in combination with H-01: a user whose tokens are permanently stranded sees a green success indicator, delaying the support escalation that might otherwise trigger a manual refund attempt while the return lane is still healthy.

■ Code

CrossChainERC4626Adapter.sol :

```
function ccipReceive(Client.Any2EVMMessage calldata message) external onlyRouter
    try this.processMessage(message) {
        emit MessageSucceeded(message.messageId);
    } catch {
        messageErrorCode[message.messageId] = ErrorCode.BASIC;
        _storeFailedMessageRecord(message.messageId, message);
        emit MessageFailed(message.messageId);
        // OffRamp still sees a successful receiver call → CCIP SUCCESS
    }
}
```

```
}
```

TrackedMessages.tsx :

```
if (status === "SUCCESS") // green - never checks messageErrorCode
```

■ Recommendation

On CCIP `SUCCESS` , also read the adapter’s message status (or subscribe to `MessageFailed` events) before showing a success state. Consider a distinct “delivered, action failed” state in the UI rather than collapsing it into either success or failure.

■ Acceptance criteria

- A frontend test that a message with `messageErrorCode == BASIC` renders as failed or partially failed, never as an unqualified success.

■ Status

Pending

[↑ Back to index](#)

I-04 | Info

Fail-open cooldown authorization bypasses the file’s own conversion rail

Field	Value
Severity	Info
Source	cross-chain-erc4626-vault/src/CrossChainERC4626Adapter.sol
ID	I-04
Prior IDs	CCA-2 (remediation brief)
Commit	cde2faffec4470c9a3f33103efd6a52d027cd696
Status	Pending

■ Description

The fork's sole authorization gate is the single location in the contract that bypasses the file's own `_normalizeToBytes32` rail and compares against a mapping whose `bytes32(0)` default doubles as a valid identity. `bytes32(bytes)` never reverts: empty input yields `bytes32(0)`, short input is right-zero-padded, and input longer than 32 bytes is silently truncated to the first word. This behaviour is identical at the pinned solc 0.8.24 and at 0.8.30.

The refund paths all use the rail — call sites `:672`, `:724`, `:744`. The authorization path does not. The contract header at `:33-35` explicitly instructs forkers to *"preserve the safety rails: ... and address conversion checks."*

■ Basis for the Info rating

The gate's safety rests entirely on every enabled source lane delivering `message.sender` as exactly 32 non-zero bytes. This holds for both supported families — EVM (`abi.encode(msg.sender)`) and SVM (32-byte Pubkey) — verified against pinned CCIP source. The gate is fail-open in orientation but is not a live weakness.

This rating does not survive the onboarding of a third chain family. `setChainType` performs no wire-width validation, so nothing structurally prevents typing a future family whose sender encoding is not 32 bytes.

■ Impact

None against the currently supported chain families. On onboarding a family whose sender encoding is not exactly 32 bytes, the gate would compare a zero-padded or truncated key against a mapping whose default is itself a valid comparison target — that is, it would authorize rather than reject. This is the same class of defect as M-02.

■ Recommendation

One line each, with no behavioural cost:

```
bytes32 senderKey = _normalizeToBytes32(message.sender);           // reverts unle
bytes32 owner = cooldownAccounts[message.sourceChainSelector][cooldownId];
if (owner == bytes32(0) || owner != senderKey) revert InvalidCooldown();
```

`senderKey` should also be used for the write at `:548` and the event at `:549`, so that a cooldown cannot be recorded under a malformed or zero principal in the first place. The `owner == bytes32(0)` guard is what converts the gate from fail-open to fail-closed and

should be retained even after normalization.

Secondary hardening. `setChainType` should reject families whose wire sender width is not 32 bytes, or `ChainType` should be widened, before any non-EVM/SVM lane is onboarded.

■ **Acceptance criteria**

- Negative tests with a zero-length and an over-length `message.sender` that revert at the gate.
- Note that `ccipReceive` 's bare `catch ( :485-486 )` collapses every failure to `ErrorCode.BASIC` , so a test routed through the Router cannot establish which check rejected. Assertions should target `processMessage` directly, or the catch should bind the error data — see L-01.

■ **Status**

Pending

[↑ Back to index](#)

I-05 | **Info**

Each order signer must separately call `setRecipientStatus`

Field	Value
Severity	Info
Source	<code>tenbin-contracts/src/.../BebopHook.sol</code> (deployment and operations)
ID	I-05
Prior IDs	CTRL-1 (remediation brief)
Commit	<code>633540ee51d7a069f4a5a53feccd8e3498d1ff14</code>
Status	Pending

■ **Description**

Each Controller order signer must individually call `setRecipientStatus(hook, true)` . No

owner, deploy script, or migration path can perform this on their behalf, so a freshly rotated signer is silently unable to transact until it performs the step itself.

■ Impact

Operational only. A signer rotation that omits the step produces a silent outage for that signer with no on-chain signal distinguishing it from any other failure.

■ Recommendation

Add `setRecipientStatus` as a mandatory deployment and rotation step in the runbook and in the NatSpec; verify the status in operational tooling; add a preflight getter or check.

■ Acceptance criteria

- A regression test establishing that a freshly rotated signer cannot transact until it has authorized the hook as a recipient.

■ Status

Pending

[↑ Back to index](#)

7. Additional observations

Recorded for completeness. These are not counted as findings and carry no severity. Two further observations from the remediation brief (`N1` , `N5`) were promoted into L-01 and I-02 respectively and do not appear here.

N-2	<code>cooldownAccounts[chain][id]</code> is never cleared following a successful UNSTAKE or CANCEL_COOLDOWN. Stale authorization records outlive the cooldowns they authorize.
N-3	<code>allocate</code> and <code>deallocate</code> sample <code>oldAllocation</code> before the external call (<code>:70</code> , <code>:97</code>), so a reentrant target observes a stale value.
N-4	Bits 136–255 of <code>actionAndId</code> are neither read nor validated (<code>:516–518</code>), presenting a forward-compatibility hazard. See M-01.
	Rate bound uses floor division. <code>BebopHook.sol:226</code> compares

N-6	<code>Math.mulDiv(outputAmount, quoteInputAmount, inputAmount) > quoteOutputAmount</code> with floor rounding, weakening the signed execution-rate bound by up to one unit.
N-7	Test-coverage gaps. (a) The cooldown authorization gate's negative tests cannot attribute the revert to that gate, owing to the bare <code>catch</code> . (b) No test exercises a pause or <code>setChainType(NONE)</code> with an outstanding cooldown. (c) No BebopHook test references the staked asset. (d) No GenericMorphoAdapter test wires the adapter through <code>setLiquidityAdapterAndData</code> . Each gap corresponds to a finding above; the acceptance criteria under I-04, M-03, L-06 and L-03 close all four.
N-8	Every vault config comments <code>max_rate</code> as ten times its actual value , on the parameter that throttles M-05. All seven read <code>max_rate = 63419583967 # 2000% APR</code> . The value is correct and is exactly <code>MAX_MAX_RATE</code> — 200% APR, the protocol maximum — and the comment is wrong by a factor of ten, so an operator reading it infers ten times more absorption headroom than the protocol will ever permit. Affected: <code>config/mainnet/{tgld,tbrl,tmxn}/vaults/*_usdc_vault.toml:22,</code> <code>config/sepolia/*/vaults/*_sepolia_vault.toml:21,</code> <code>config/local/local_vault.toml:21.</code>

8. Not found / out of scope

The following attack paths were examined and did not yield findings. They are recorded so that the negative result is auditable.

Checked	Result
Steal another user's cooldown / silo via CCIP	Not found — <code>cooldownAccounts</code> binds sender; <code>onlyRouter</code> ; <code>processMessage</code> atomic. Orientation caveat recorded as I-04
Profitable inflation / precision drain against configured targets	Not found — dead deposits; donation subsidizes; not found on live Morpho path. Class-level exposure recorded as L-05
Half-committed unstake + failed bridge	Rejected — <code>try this.processMessage</code> atomicity
Controller / Bebop / EIP-712 order theft	Not found — order hash + nonce + signer + recipient + oracle bounds
Custodian / Silo / Morpho parent auth	Not found — role / <code>parentVault</code> gated; no unauthorized path found

drain	
<code>refundFailedMessage</code> steal to attacker	Not found — tokens always return to original sender; permissionless trigger is intentional
Bebop hook / router broken redeem	Not a finding — confirmed expected behavior; router path works
Administrative drain of outstanding cooldowns	Not found — no <code>rescueTokens</code> ; <code>withdrawFee</code> deleted in the fork; <code>recoverNative</code> moves native only. This absence is also what removes the escape hatch in M-03
<code>forceDeallocate</code> blocked by the deployed <code>Gate</code>	Rejected — the gates do not constrain adapter-to-idle movement; demonstrated under M-04

9. Assumptions

The following are carried as assumptions and are not established by this review. Findings rest on them, and Tenbin should confirm or correct each.

Assumption	If incorrect
No future CCIP family will be typed as <code>EVM</code> or <code>SVM</code> in <code>setChainType</code>	I-04's containment argument weakens; the rating rises above Info
Chainlink's lane-retirement policy provides notice sufficient to drain outstanding cooldowns	M-03 escalates from a temporary freeze to permanent loss
The OffRamp treats a <code>ccipReceive</code> that returns successfully after a caught failure as ineligible for manual re-execution	The message-burn component of L-01 is void; the bookkeeping component stands
Bebop's <code>needsApproval</code> leaves no residual router-to-hook allowance across transactions	The hook's only partial-fill guard is unsound

No chain state was read during this review. Every statement regarding what is deployed, configured, or current is a claim about the repository snapshot at the reviewed commits.

10. Client determinations required

#	Item	Bears on
1	The scope wording in §3 — whether “non-synchronous ERC4626” is literal or typographical	H-02 severity, and the scope of remediation item 2 in §5
2	Confirmation or correction of the assumptions in §9	As tabulated
3	Whether sUSDS (0xa3931d71...) or Aave stataUSDC (0xD4fa2D31...) carry an admin-settable exit fee. Not verified during this review; settled by a block-explorer check of the proxy admin	M-04 reachability. The Medium rating does not depend on it
4	Whether GenericMorphoAdapter is intended to be set as liquidityAdapter. It is absent from <code>script/</code> and <code>config/</code> at present	If not intended, L-03 reduces to a NatSpec requirement and L-05’s automatic-allocation variant is eliminated. If intended, L-03 should be re-rated to Medium and both require full remediation
5	Whether a zero beneficiary is intended on either chain family	M-02. If intended, the finding reduces to a documentation item; if not, the SVM return path requires a guard
6	Whether any privileged flow resolves adapters through genericMorphoAdapter[parent][vault]	L-07. The current rating assumes the registry is a convenience lookup only; if privileged flows depend on it, re-rate

11. Remediation review

Findings will be re-examined against the acceptance criteria stated in §6 once remediation is applied, together with a check that no fix has introduced a new defect — in particular the M-05 patch, whose naive form creates a fund lock, and the L-07 patch, whose naive form creates a squatting denial of service.

H-01 must be resolved before deploy. It is the only finding in this report that is unfixable on the live contract, and remediation item 1 in §5 is therefore a release gate rather than a queue item.

Conditional severities will be finalized upon receipt of the determinations in §10. The public

report will be issued against the remediated code.

Appendix A — Consolidation record

This report merges two working documents, each of which had itself consolidated earlier material. The full lineage is recorded below for traceability.

A.1 — Source documents

Document	Date	Scheme	Findings
tenbin-security-audit-consolidated.md	—	H/M/L/I- NN	13 (itself merging tenbin-security-audit_0812.md , findings.md , findings-audit-tenbin-2026-08.md)
TENBIN_REMEDIATION_BRIEF_2026-08-15.md	2026-08-15	GMA-* , CCA-* , BH-* , GMAF-* , CTRL-* , N*	11 findings + 8 observations

A.2 — Disposition

Prior ID	Source	This report	Treatment
H-01	consolidation	H-01	Unchanged; corroborating movement-site enumeration added from CCA-1
M-01	consolidation	M-01	Unchanged; N4 cross-reference added
M-02	consolidation	M-02	Unchanged; cross-reference to CCA-2 added
M-03	consolidation	L-03	De-escalated to Low; merged with GMA-1 ; reconciliation recorded in-finding

L-01	consolidation	L-01	Merged with N1 ; message-burn mechanic and error-binding recommendation added
L-02	consolidation	L-02	Unchanged; acceptance criteria added
I-01	consolidation	M-03	Escalated to Medium; merged with CCA-1 and I-06 ; PoC retained
I-02	consolidation	I-01	Renumbered; cross-reference to GMA-2a gate analysis added
I-03	consolidation	I-02	Renumbered; merged with N5
I-04	consolidation	I-03	Renumbered; acceptance criteria added
I-05	consolidation	H-02	Escalated to conditional High; merged with GMA-4
I-06	consolidation	M-03	Merged into M-03 as the source-chain-binding aspect
I-07	consolidation	L-07	Escalated to Low; merged with GMAF-1
GMA-4	brief	H-02	Renumbered; consolidation's Info rating reconciled
CCA-1	brief	M-03	Renumbered; absorbed I-01 and I-06
GMA-2a	brief	M-04	Renumbered; unchanged
GMA-2b	brief	M-05	Renumbered; unchanged
CCA-3	brief	L-04	Renumbered; unchanged
GMA-1	brief	L-03	Renumbered; merged with M-03
GMA-3	brief	L-05	Renumbered; unchanged
BH-1	brief	L-06	Renumbered; impact section added
GMAF-1	brief	L-07	Renumbered; absorbed I-07 's impact analysis and event recommendation
CCA-2	brief	I-04	Renumbered; impact section added
CTRL-1	brief	I-05	Renumbered; impact section added

N1	brief	L-01	Merged
N5	brief	I-02	Merged
N2 , N3 , N4 , N6 , N7 , N8	brief	§7	Retained as observations; renumbered N-*

A.3 — Coverage differences between the source documents

Recorded because the two documents had materially different scopes, and the union is what this report delivers.

Finding	Present only in	Note
H-01	consolidation	The sole unconditional High. The brief analysed the same recovery functions under <code>CCA-1</code> but reasoned only about the tokenless case, and did not reach the token-carrying failure with a zero <code>localRefundAddress</code> .
M-01, M-02	consolidation	Payload-encoding and beneficiary-validation findings in <code>CrossChainERC4626Adapter</code> , which the brief covered in depth on other axes.
L-02, I-03	consolidation	Frontend was outside the brief's stated contract scope.
I-01	consolidation	<code>Gate.sol</code> was outside the brief's stated contract scope.
M-04, M-05, L-05	brief	<code>GenericMorphoAdapter</code> valuation and inflation findings, including upstream audit provenance (<code>CS-MORPHO-VLT2-020</code>).
L-04, I-04	brief	Native-reserve exhaustion and cooldown authorization orientation.
L-06, I-05	brief	<code>BebopHook</code> was outside the consolidation's stated component list.

A.4 — Open items before delivery

1. **Commit hashes.** L-02, I-01 and I-03 carry no commit hash in the source material. Populate before delivery.

2. **Status fields.** All findings are marked Pending. Update as remediation lands.
3. **Severity finalization.** H-02 and L-03 carry conditional ratings pending determinations 1 and 4 in §10.
4. **Proof-of-concept coverage.** Only M-03 carries an executable PoC. M-04 item 1 and M-05 are asserted as demonstrated but ship no test; both should carry one before the public report, since each is a Medium resting on a claim about permissionless reachability.
5. **Solidity versions.** The brief records 0.8.24 / 0.8.27 / 0.8.30 across the reviewed set. Confirm the per-contract mapping before publication.